• @ConditionalOnProperty Annotation :-

@ConditionalOnProperty :- Bean is created Conditionally (means Bean can be created or Not.

@PostConstruct :- ~~After this initilized Before Bean is~~ ~~Start~~ successfully ~~created~~

@PostConstruct :- Spring calls the methods annotated with @PostConstruct only once just after the initialization of bean properties.

```
@Component
Public class DBConnections{
    @Autowire
    MySQLConnection mySQLCon;

    @Autowire
    NoSQLConnection NoSQLCon;

    @PostConstruct
    Public void init(){
        S.O.P("DB connection Bean Created with dependencies")
        S.O.P(Object.isNull(mySQLCon));
        S.O.P(Object.isNull(noSQLCon));
    }
}
```

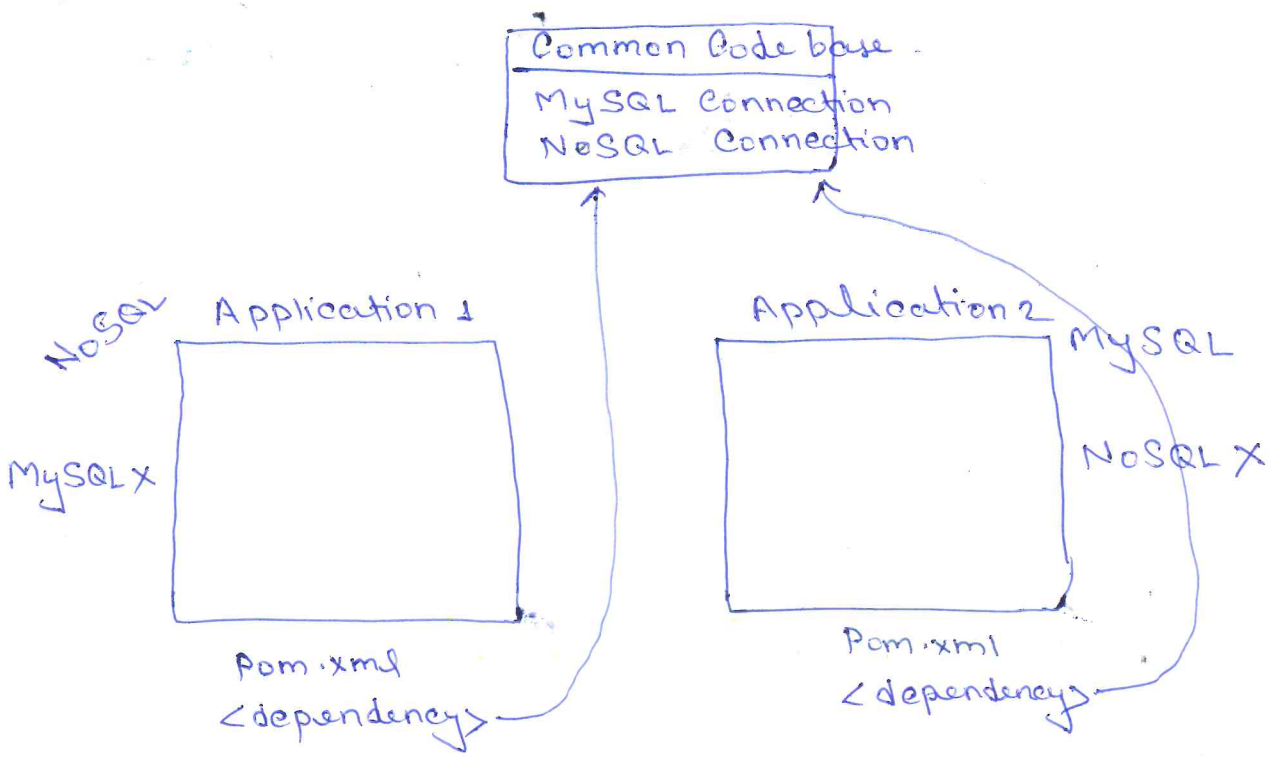
```
@Component
Public class NoSQLConnection{
    NoSQLConnection(){
        S.O.P("initialization of NoSQL");
    }
}
```

```
@Component
Public class MySQLConnection{
    MySQLConnection(){
        S.O.P("mySQL");
    }
}
```

Concept && Coding

Use Case 1: We want to Create only 1 Bean, either MySQLConnection or NoSQLConnection.

Use Case 2: We have 2 Components, sharing same codebase, But 1 Component need MySQLConnection and other needs NoSQLConnection.



Concept && Coding

Solution is @ConditionalOnProperty Annotation ③

```
@Component
@ConditionalOnProperty(prifix = "SQLConnection", value = "enabled", havingValue = "tr
                                  ①                  ②                  ③
                                  matchIfMissing = false)
                                  ④
Public class MySQLConnection{
    S.o.p("initialization of MySQLConnection Bean");
}
}
```

```
@Component
@ConditionalOnProperty(prifix = "NosqlConnection", value = "enabled", havingvalue =
                                  ①                  ②                  ③
                                  ; matchIfMissing = false
                                  ④
Public class NoSQLConnection{
    NoSQLConnection(){
        S.o.p("initialization of NoSQLConnectionBean");
    }
}
```

Application.properties

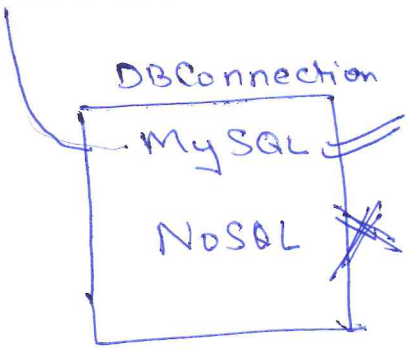
```
sqlconnection.enabled = true
```

```
@Component
Public class DBConnection{
    @Autowired(required = false)
    MySQLConnection mySQLCon;

    @Autowired(required = false)
    NoSQLConnection NoSQLCon;

    @PostConstruct
    Public void init(){

}
```



Concept && Coding

Advantage

1 Toggling of Feature
2: Avoid cluttering Application context with un-necessary beans
3; Save Memory
4: Reduce application Startup time.

Disadvantages :

1 : Misconfiguration can happen
2 ;- Code Complexity when over used.
3 ;. Multiple bean creation with same configuration, brings confusion.
4 . Complexity is managing.

Concept && Coding